

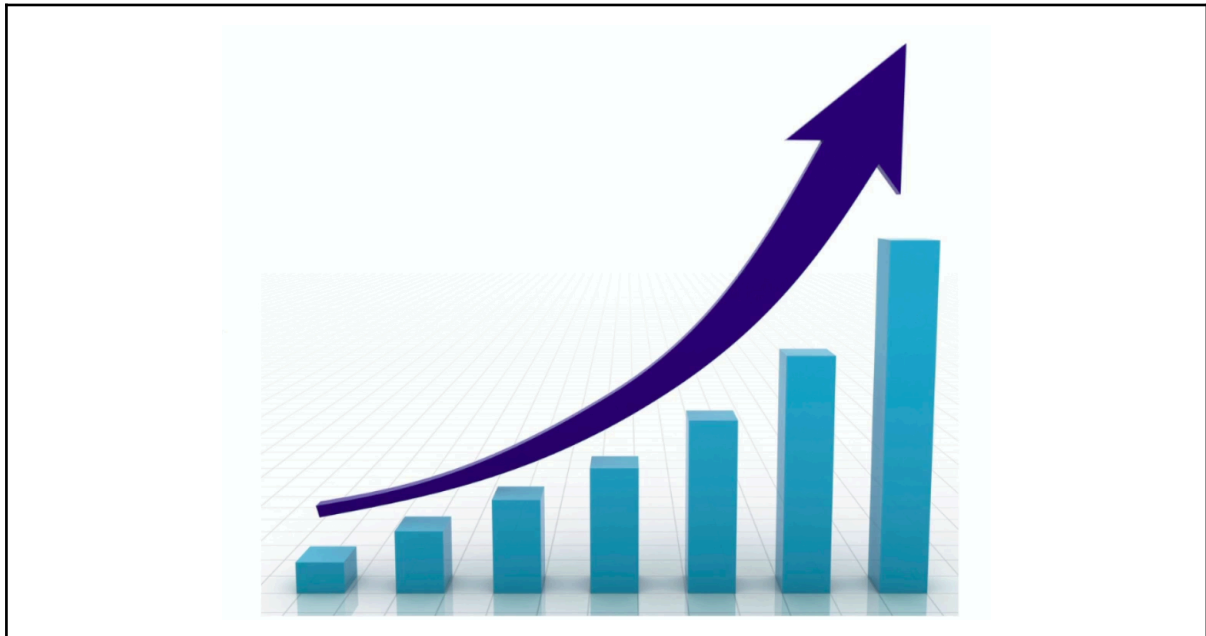
Section 6: Scalability in System Design

Introduction to scalability

In this lecture, we will explore one of the most important goals of system design, building systems that can grow gracefully as users, traffic, and data continue to increase. So let's get started.

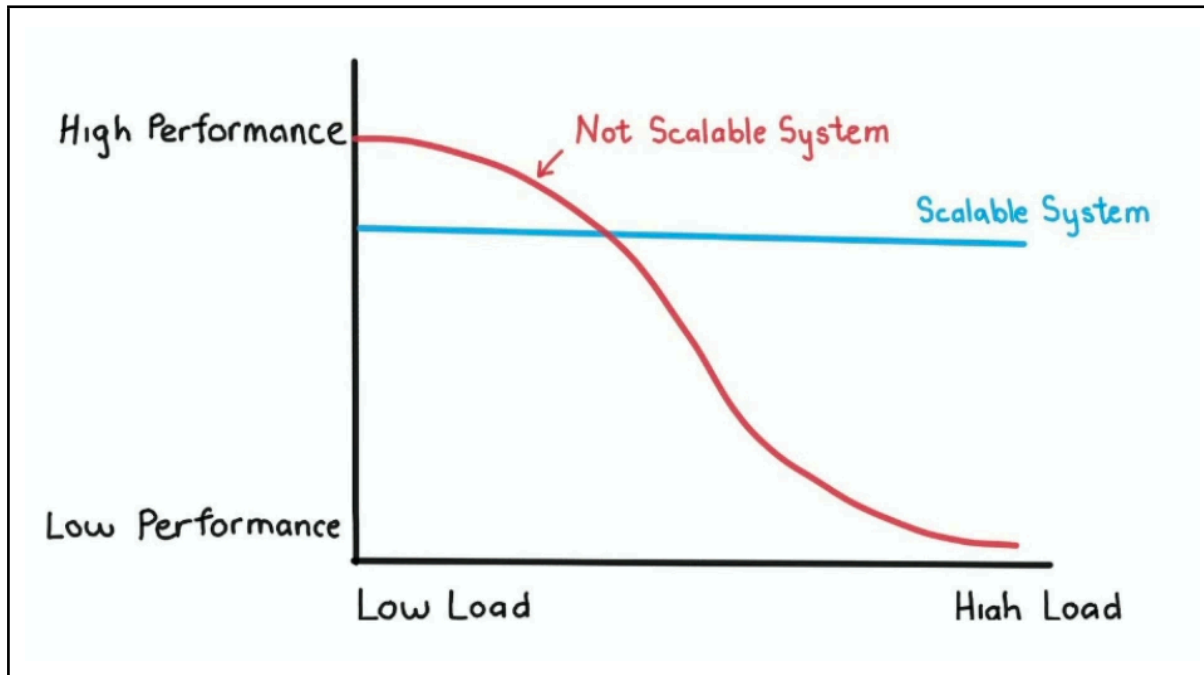
What is scalability?

Scalability is one of the most important qualities of a well-designed system because success almost always brings growth. A system that works perfectly for a thousand users may struggle when the number becomes a hundred thousand or even million. At its core scalability is not just about handling more users. It is about maintaining a good user experience as demand increases, as traffic grows, the system should continue to deliver acceptable performance, it should remain reliable, and stay available without requiring a completed redesign.



Think of scalability as the system's ability to grow with the business. Growth can come in many forms, more users, more requests, more data, or more transactions. A scalable architecture anticipates that growth and provides a path to handle it efficiently. From an architect's perspective, the key question is never whether the system works today. The real question is what happens when demand becomes ten times larger. If performance degrades, costs explode, or reliability suffers, the design is not truly scalable. Developing systems that can evolve gracefully under increasing load is one of the fundamental goals of system design.

Why do systems need to scale?

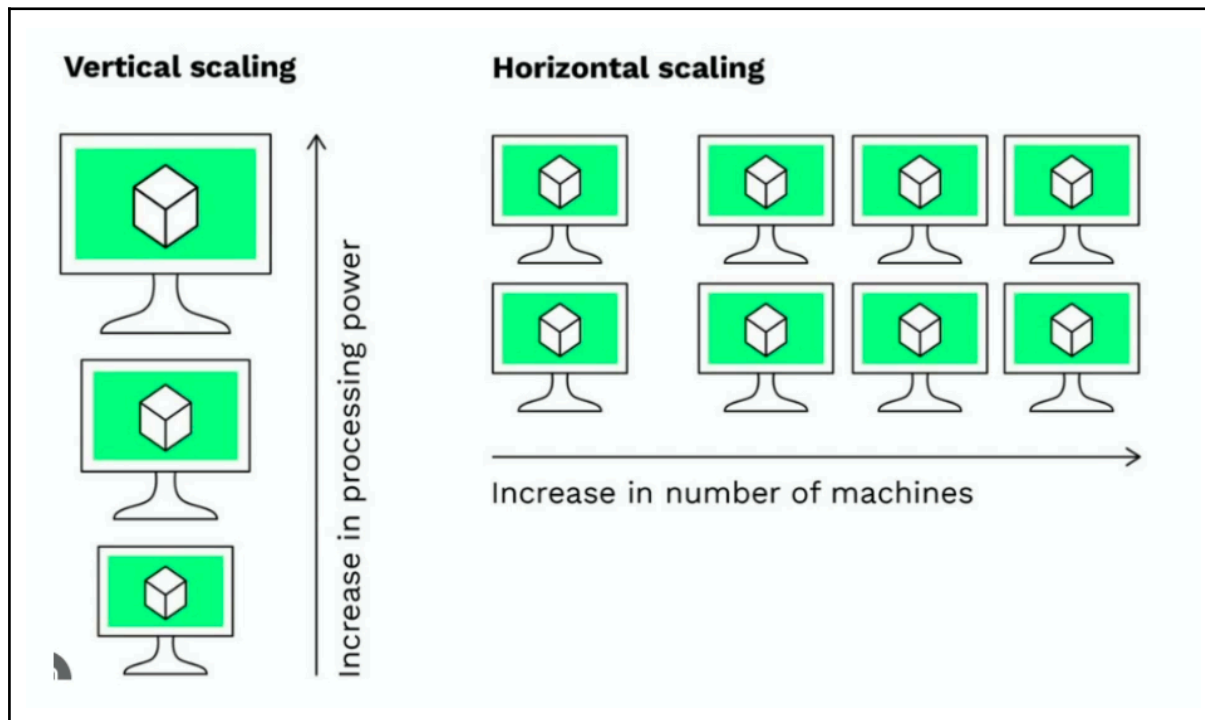


Systems rarely fail due to normal traffic, they fail when growth or sudden demand exceeds what the architecture was designed to handle. The below are some of the drivers that causes system failures due to bad scalable design:

- **User growth:** A product may start in a single market, but as it expands to new regions, the number of concurrent users can increase dramatically. What worked for thousands of users may not work for millions.
- **Data growth:** Modern applications continuously generate logs, analytics, events, transactions and IoT data. Overtime, storing, processing, and querying that data becomes a scalability problem of its own.
- **Peak loads:** Events like black friday sales, ticket releases, or viral content can create massive surges within minutes. These peak loads often expose bottlenecks that remain hidden during normal operations.

Beyond handling growth, scalability protects user experience. As demand increases, users still expect fast response time and reliable services. Many organisations are bound by performance and availability SLAs too, making scalability a business requirement rather than just a technical concern. Ultimately, scalability is what allows a system to grow without sacrificing performance, reliability, or customer trust. It turns success from a risk into an opportunity.

Types of Scalability



At a very high level, there are 2 primary ways to scale a system, and almost every architecture that you encounter will rely on one or both of them. The types of scalability are:

- **Vertical Scaling:** In vertical scaling we increase the capacity of a single machine by adding more CPU, memory usage, storage, or network resources. It is often the simplest way to handle additional load because the application architecture usually remains unchanged.
- **Horizontal Scaling:** In horizontal scaling we add more machines and distribute traffic across them. Instead of building a bigger server, we build a larger system of servers working together. This approach is what enables internet-scale platforms to support massive growth.

Neither approach is universally better. Vertical scaling offers simplicity, while horizontal scaling provides greater growth potential and resilience. Experienced architects evaluate both based on traffic patterns, operational complexity, cost and long-term scalability goals. Simply to understand, vertical scaling means a bigger machine, while horizontal scaling means more machines.

Common challenges in scaling

As systems grow, handling more traffic is only part of the challenge. The real difficulty is managing the side effects that come with that scale. The below are some of the most common challenges that comes with scalability:

1. **Latency:** Every network call, database query and service-to-service interaction add delay. In distributed architecture, a single user request may pass through multiple services before a response is returned, making latency a critical design concern.
2. **Bottlenecks:** No matter how scalable the overall architecture looks, one overloaded component can limit the performance of the entire system. A busy database, a resource-constrained service, or a sequential processing step can become the point where growth starts to break down.
3. **Downtime:** Scaling also increases the risk of downtime. More servers, services and dependencies. All this means more potential failure points. Deployments, infrastructure changes and scaling events must be carefully managed to maintain high availability.
4. **Cost:** Every scalability decision has a financial impact. Adding capacity improves performance, but excessive scaling can quickly drive infrastructure costs beyond what the business can justify.

This is why scalability is fundamentally a balancing act. And as architects, we are not trying to maximize scale at any cost. We are trying to achieve the right balance between performance, reliability, availability, and cost.

Summary & What's Next

- Scalability helps systems grow without breaking
- Two main types: vertical and horizontal
- But it introduces latency, cost, and other risks
- What's Next:
 - Scaling Strategies — how to actually implement horizontal, vertical, and diagonal scaling with trade-offs and real-world choices

Interview Questions - Scaling

- What does scalability mean in the context of system design?
- Can you explain a real-world example where scalability was critical to success or failure?
- What are the main challenges systems face as they scale?
- How would you identify a bottleneck in a scalable architecture?
- Why does latency increase with scale, and how can you mitigate it?
- How do you balance scalability with cost in cloud-based systems?